МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

Національний технічний університет «Дніпровська політехніка»

Кафедра Системного аналізу та управління

ЗВІТ

З лабораторної роботи

З дисципліни «Поглиблене програмування в середовищі Java»

Виконав:

студент групи 122-22-5

Вієнко В.Ю.

Перевірив: доцент

Мінєєв Олександр Сергійович

м. Дніпро

2026 рік

Тема: Розробка REST-застосунку на Spring Boot.

Мета: Розробити повноцінний клієнт-серверний застосунок на мові Java, що реалізує бізнес-логіку моніторингу криптоактивів, автоматизацію збору даних через зовнішні API та управління портфелем користувача.

Завдання:

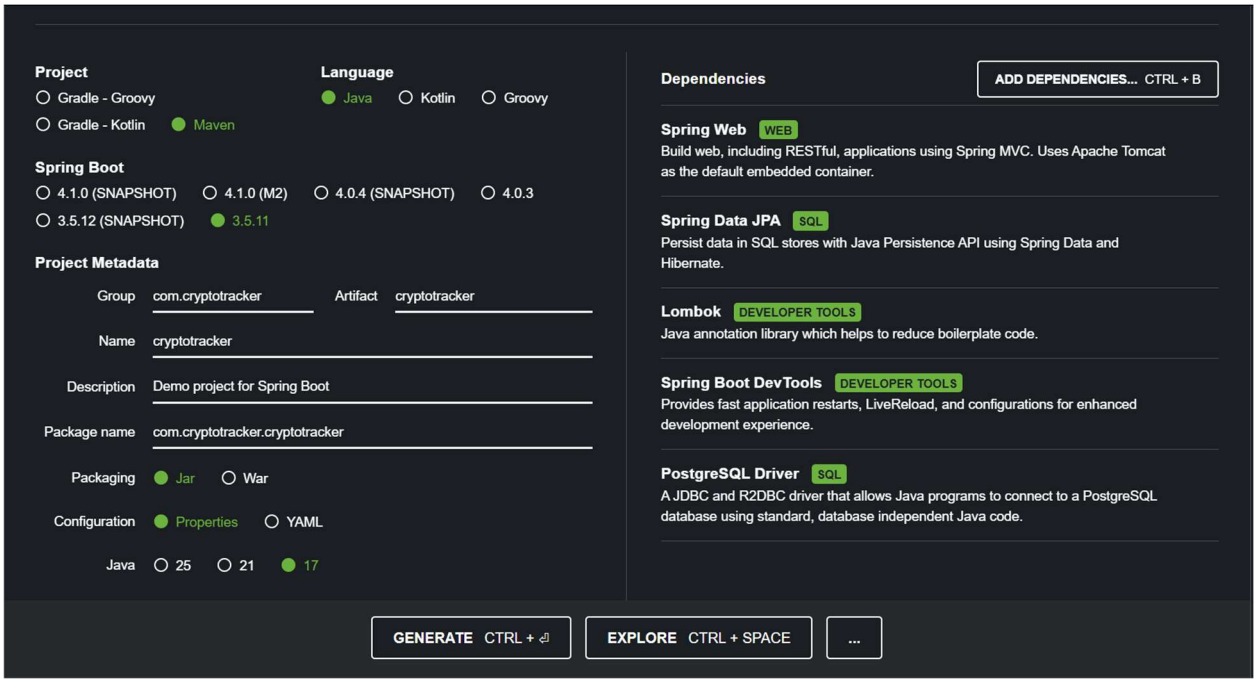
1. Налаштувати стек технологій: Java, Spring Boot 3.x, H2 Database (File-based).
2. Реалізувати серверне відображення за допомогою Thymeleaf та клієнтську логіку (Vanilla JS, Bootstrap 5).
3. Побудувати реляційну базу даних з 3 пов'язаними таблицями (Users, Articles, Categories).
4. Розробити фоновий модуль парсингу даних з використанням бібліотеки Jsoup.
5. Реалізувати систему безпеки та розсилки Email-повідомлень (Spring Mail) для верифікації акаунтів.
6. Реалізувати повний цикл CRUD та пагінацію для виводу новин.

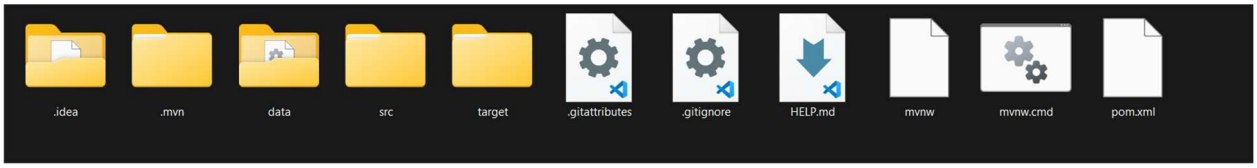
Хід роботи

1. **Створили та завантажили шаблон для розробки проєкту:**

Виконано ініціалізацію Spring Boot проєкту. Додано основні залежності у pom.xml: Spring Web, Spring Data JPA, H2 Database, Jsoup (для парсингу HTML/XML), Spring Boot Mail (для пошти) та Thymeleaf. Налаштовано змінні середовища (Environment Variables) в IntelliJ IDEA для безпечного зберігання пароля від поштового сервера Gmail, щоб уникнути витоку

конфіденційних даних у систему контролю версій Git.





2. Створення моделей даних (Entity):

Розроблено сутності бази даних з використанням анотацій JPA:

- Category.java – описує категорію новин.
- Article.java – описує новину (заголовок, контент, посилання, зображення). Зв’язана з категорією через @ManyToOne.
- User.java – модель користувача для авторизації. Містить поля password, isVerified та verificationCode. Зв’язана з новинами через @ManyToMany (для реалізації функціоналу "Закладки").

```java
@Entity  Volodymyr Vienko
@Table(name = "articles")
public class Article {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(length = 500)  3 usages
    private String title;

    @Column(columnDefinition = "TEXT")  3 usages
    private String content;

    @Column(length = 2000, nullable = false)  3 usages
    private String sourceUrl;

    @Column(length = 1000)  4 usages
    private String imageUrl;

    private LocalDateTime publishedAt;  3 usages

    @ManyToOne(fetch = FetchType.LAZY)  3 usages
    @JoinColumn(name = "category_id")
    private Category category;

    public Article() {}  no usages  Volodymyr Vienko
```

```java
@Entity  Volodymyr Vienko
@Table(name = "categories")
public class Category {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)  3 usages
    private String name;

    @OneToMany(mappedBy = "category", cascade = CascadeType.ALL, orphanRemoval = true)  2 usages
    private List<Article> articles = new ArrayList<>();

    public Category() {}  no usages  Volodymyr Vienko

    public Category(String name) { this.name = name; }

    public Long getId() { return id; }  1 usage  Volodymyr Vienko
    public void setId(Long id) { this.id = id; }  no usages  Volodymyr Vienko
    public String getName() { return name; }  Volodymyr Vienko
    public void setName(String name) { this.name = name; }  Volodymyr Vienko
    public List<Article> getArticles() { return articles; }  no usages  Volodymyr Vienko
    public void setArticles(List<Article> articles) { this.articles = articles; }  no usages  Volodymyr Vienko
}
```

```java
@Table(name = "users")
public class User {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)  3 usages
    private String username;

    @Column(nullable = false)  3 usages
    private String password;

    private boolean isVerified = false;  2 usages

    private String verificationCode;  2 usages

    @Column(nullable = false, unique = true)  3 usages
    private String email;

    @ManyToMany(fetch = FetchType.LAZY)  4 usages
    @JoinTable(
            name = "user_saved_articles",
            joinColumns = @JoinColumn(name = "user_id"),
            inverseJoinColumns = @JoinColumn(name = "article_id")
    )
    private Set<Article> savedArticles = new HashSet<>();

    public User() {}  no usages  Volodymyr Vienko
```

3. Налаштування доступу до даних (Repositories):

Створено інтерфейси ArticleRepository, CategoryRepository та UserRepository, які наслідують JpaRepository. Реалізовано кастомні методи пошуку:

- Пагінація та пошук новин за текстом або заголовком (findByTitleContainingIgnoreCaseOrContentContainingIgnoreCase).
- Фільтрація новин за категорією (findByCategory_NameIgnoreCase).
- Пошук користувача за поштою (findByEmail).

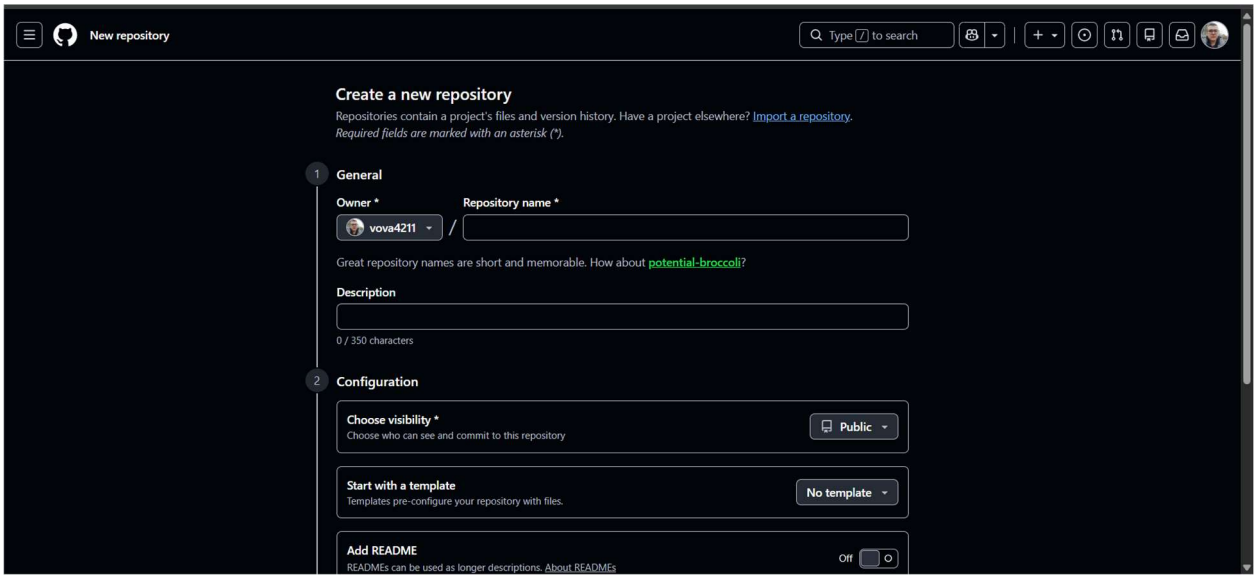
```
import org.springframework.data.domain.Pageable;

@Repository  9 usages  Volodymyr Vienko
public interface ArticleRepository extends JpaRepository<Article, Long> {
    Page<Article> findByTitleContainingIgnoreCaseOrContentContainingIgnoreCase(String title, String content, Pageable pageable);  1 usage  Volodymyr Vienko

    boolean existsBySourceUrl(String sourceUrl);  1 usage  Volodymyr Vienko

    Page<Article> findByCategory_NameIgnoreCase(String categoryName, Pageable pageable);  1 usage  Volodymyr Vienko
}
```

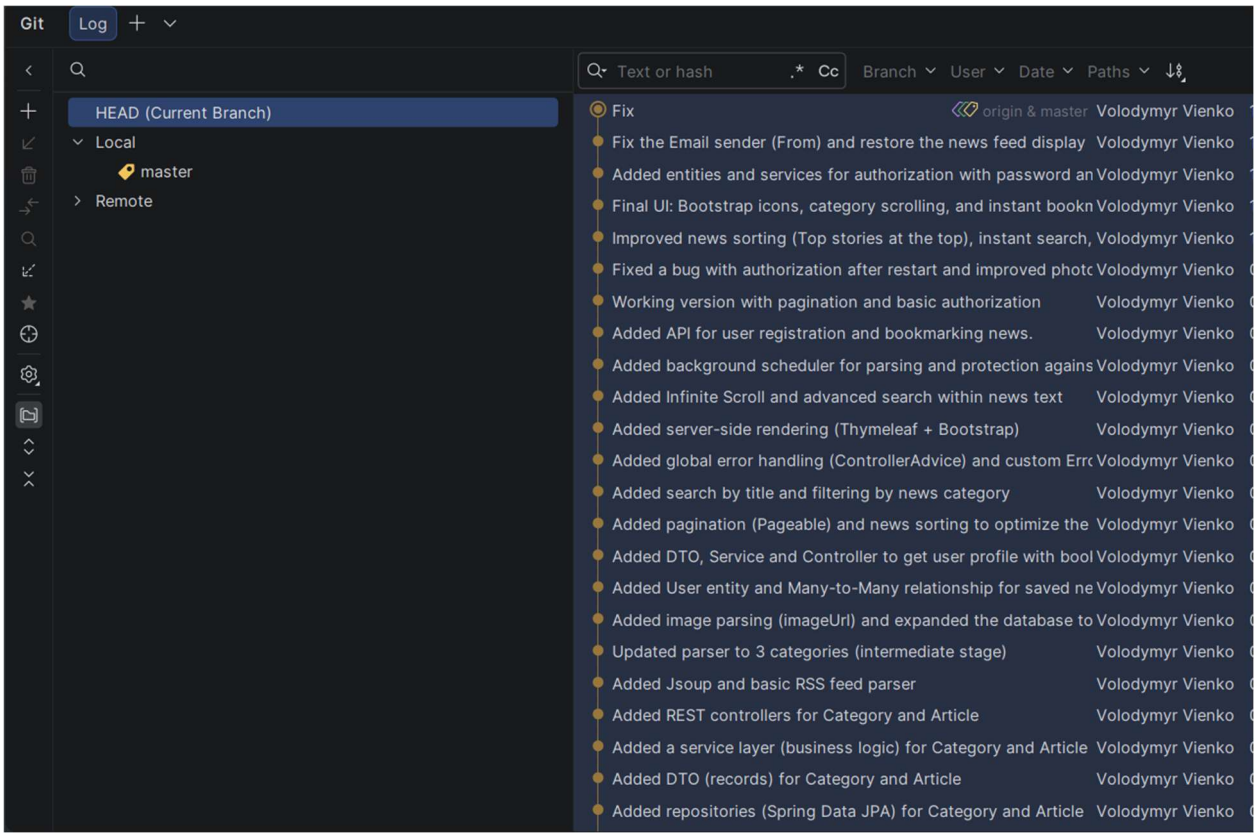
String categoryName
news-aggregator

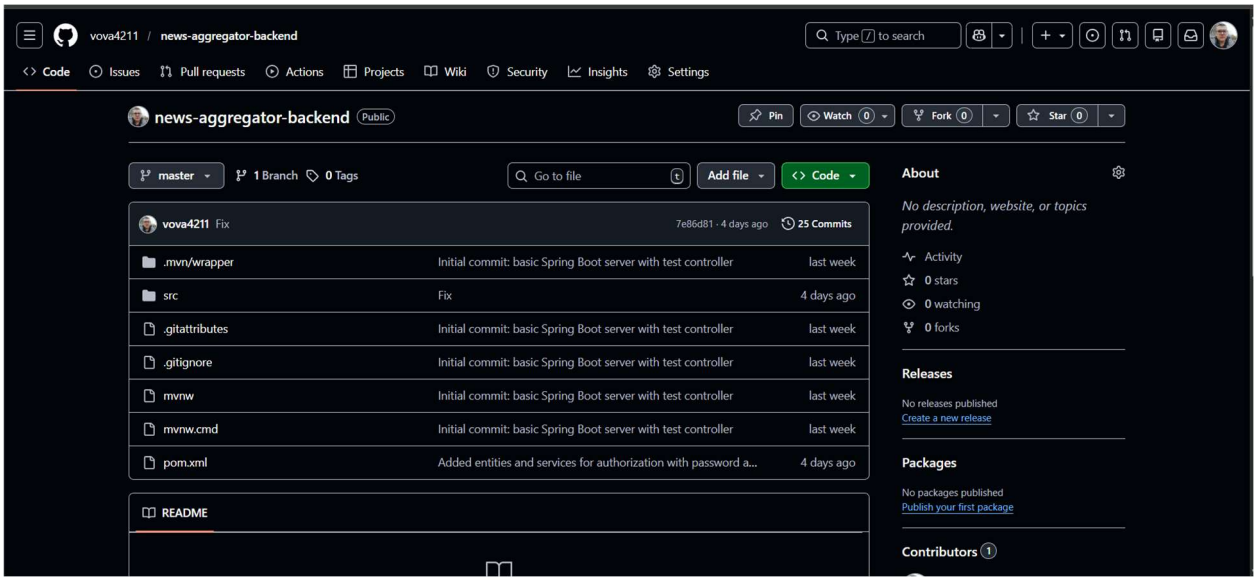
```
@Repository  6 usages  Volodymyr Vienko
public interface CategoryRepository extends JpaRepository<Category, Long> {

    Optional<Category> findByName(String name);  1 usage  Volodymyr Vienko
}
```

```
@Repository  3 usages  Volodymyr Vienko
public interface UserRepository extends JpaRepository<User, Long> {
    Optional<User> findByEmail(String email);  5 usages  Volodymyr Vienko
}
```

4. Створили репозиторій GitHub та підключили його до проєкту:







Посилання на репозиторій: https://github.com/vova4211/news-aggregator-backend.

5. Розробка бізнес-логіки (Services):

- **NewsParserService**: Реалізовано фоновий збір даних (за допомогою анотації @Scheduled). Парсер звертається до RSS-стрічок топових видань, витягує зображення та зберігає нові статті в БД.

- **CategoryService**: Керує логікою отримання списку всіх доступних категорій для відображення в меню.
- **ArticleService**: Забезпечує сортування, пагінацію новин та їх конвертацію у ArticleDto.
- **EmailService**: Налаштовано інтеграцію з SMTP сервером. Генерує та надсилає користувачам 6-значні коди підтвердження при реєстрації.
- **UserService**: Керує безпекою (перевірка паролів, валідація кодів з пошти) та функціоналом збереження новин у закладки за допомогою транзакцій (@Transactional).

```
@Service  Volodymyr Vienko
public class NewsParserService {

    private final CategoryRepository categoryRepository;  3 usages
    private final ArticleRepository articleRepository;  3 usages

    public NewsParserService(CategoryRepository categoryRepository, ArticleRepository articleRepository) {  no usages  Volodymyr Vienko
        this.categoryRepository = categoryRepository;
        this.articleRepository = articleRepository;
    }

    @PostConstruct  Volodymyr Vienko
    @Scheduled(fixedDelay = 600000)
    public void parseNews() {
        try {
            System.out.println("Починаю фоновий збір новин...");

            String[][] feeds = {
                    {"Спорт", "http://feeds.bbci.co.uk/sport/rss.xml"},
                    {"Спорт", "https://rss.nytimes.com/services/xml/rss/nyt/Sports.xml"},
                    {"Спорт", "https://www.espn.com/espn/rss/news"},
                    {"Ігри", "https://www.polygon.com/rss/index.xml"},
                    {"Ігри", "https://www.ign.com/rss/articles/feed"},
                    {"Розваги", "http://feeds.bbci.co.uk/news/entertainment_and_arts/rss.xml"},
                    {"Розваги", "https://www.buzzfeed.com/entertainment.xml"},
                    {"Здоров'я", "http://feeds.bbci.co.uk/news/health/rss.xml"},
                    {"Освіта", "http://feeds.bbci.co.uk/news/education/rss.xml"},
                    {"Наука", "http://feeds.bbci.co.uk/news/science_and_environment/rss.xml"},
```

```
public class EmailService {

    private final JavaMailSender mailSender;  2 usages

e Alt+7  lue("${spring.mail.username}")
    private String senderEmail;

    public EmailService(JavaMailSender mailSender) { this.mailSender = mailSender; }

    public void sendVerificationCode(String toEmail, String code) {  1 usage  Volodymyr Vienko
        SimpleMailMessage message = new SimpleMailMessage();

        message.setFrom(senderEmail);

        message.setTo(toEmail);
        message.setSubject("Код підтвердження | News Aggregator");
        message.setText("Вітаємо у нашому стартапі!\n\n" +
                "Ваш код підтвердження: " + code + "\n\n" +
                "Нікому його не передавайте.");

        mailSender.send(message);
    }
}
```

```
@Service  Volodymyr Vienko
public class UserService {

    private final UserRepository userRepository;  9 usages
    private final ArticleRepository articleRepository;  2 usages
    private final EmailService emailService;  2 usages

    public UserService(UserRepository userRepository, ArticleRepository articleRepository, EmailService emailService) {  no usages  Volodymyr Vienko
        this.userRepository = userRepository;
        this.articleRepository = articleRepository;
        this.emailService = emailService;
    }

    @Transactional(readOnly = true)  3 usages  Volodymyr Vienko
    public UserDto getUserByEmail(String email) {
        return userRepository.findByEmail(email) Optional<User>
                .map( User user -> new UserDto(
                        user.getId(),
                        user.getUsername(),
                        user.getEmail(),
                        user.getSavedArticles().stream() Stream<Article>
                                .map( Article article -> new ArticleDto(
                                        article.getId(),
                                        article.getTitle(),
                                        article.getContent(),
                                        article.getSourceUrl(),
                                        article.getImageUrl(),
```

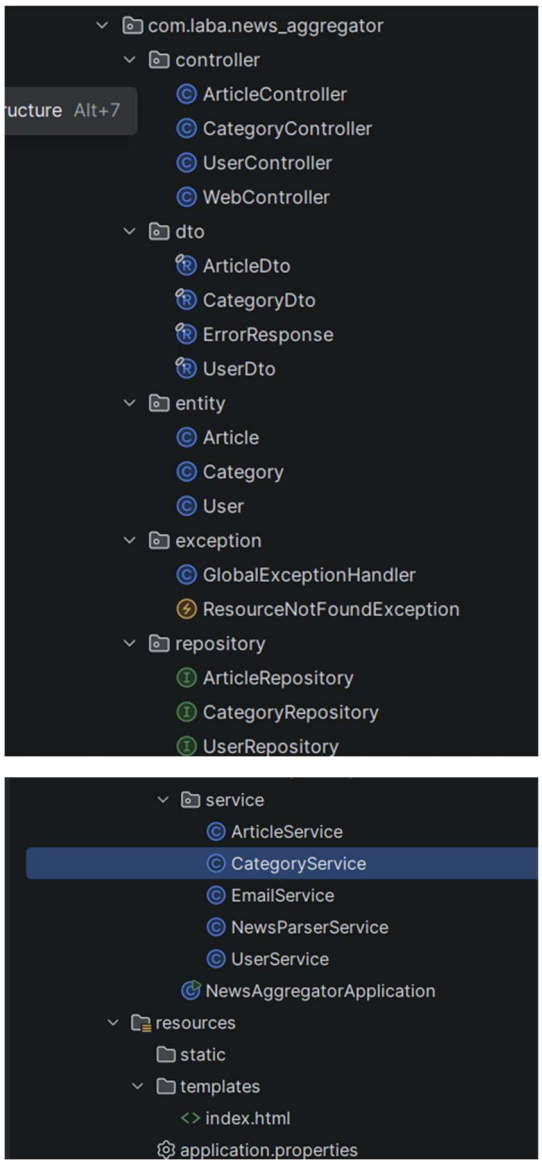
```
@Service  Volodymyr Vienko
public class ArticleService {

    private final ArticleRepository articleRepository;  4 usages

    public ArticleService(ArticleRepository articleRepository) { this.articleRepository = articleRepository; }

    @Transactional(readOnly = true)  2 usages  Volodymyr Vienko
    public Page<ArticleDto> getArticlesPaginated(int page, int size, String search, String category) {
        Pageable pageable = PageRequest.of(page, size, Sort.by( properties: "id").descending());

        Page<Article> articlePage;

        if (search != null && !search.isBlank()) {
            articlePage = articleRepository.findByTitleContainingIgnoreCaseOrContentContainingIgnoreCase(search, search, pageable);
        } else if (category != null && !category.isBlank()) {
            articlePage = articleRepository.findByCategory_NameIgnoreCase(category, pageable);
        } else {
            articlePage = articleRepository.findAll(pageable);
        }

        return articlePage.map( Article article -> new ArticleDto(
                article.getId(),
                article.getTitle(),
                article.getContent(),
                article.getSourceUrl(),
                article.getImageUrl(),
                article.getPublishedAt(),
```

```
@Service  Volodymyr Vienko
public class CategoryService {

    private final CategoryRepository categoryRepository;  2 usages

    public CategoryService(CategoryRepository categoryRepository) { this.categoryRepository = categoryRepository; }

    @Transactional(readOnly = true)  2 usages  Volodymyr Vienko
    public List<CategoryDto> getAllCategories() {
        return categoryRepository.findAll() List<Category>
                .stream() Stream<Category>
                .map( Category category -> new CategoryDto(category.getId(), category.getName())) Stream<CategoryDto>
                .toList();
    }
}
```

6. Об'єкти передачі даних (DTO) та Обробка помилок (Exceptions):

Для безпечної передачі даних між сервером та клієнтом створено спеціальні класи в пакеті dto (ArticleDto, CategoryDto, UserDto, ErrorResponse). Також

реалізовано пакет exception із класом GlobalExceptionHandler (анотований @ControllerAdvice), який перехоплює помилки (наприклад, ResourceNotFoundException) та повертає їх на фронтенд у зручному JSON-форматі (ErrorResponse).



```
public record ArticleDto(  8 usages  Volodymyr Vienko
        Long id,  no usages
        String title,  no usages
        String content,  no usages
        String sourceUrl,  no usages
        String imageUrl,  no usages
        LocalDateTime publishedAt,  no usages
        String categoryName  no usages
) {
}
```

```
public record CategoryDto(Long id, String name) {  5 usages
}
```

```
public record ErrorResponse(  7 usages
        int status,  no usages
        String message,  no usages
        LocalDateTime timestamp  no usages
) {
}
```

```
public record UserDto(  9 usages
        Long id,  no usages
        String username,  no usages
        String email,  no usages
        List<ArticleDto> savedArticles  no usages
) {
}
```

7. Розробка контролерів (Controllers) та REST API:

Створено архітектуру контролерів для обробки запитів:

- WebController – відповідає за початковий рендеринг головної сторінки (index.html), передачу початкового пулу новин та словника векторних іконок для категорій.
- ArticleController, CategoryController, UserController – повноцінні REST-контролери. Вони приймають асинхронні запити (fetch API) від клієнта у форматі JSON для підвантаження новин при прокрутці, логіну, реєстрації та роботи із закладками.

```java
@Controller  Volodymyr Vienko
public class WebController {

    private final ArticleService articleService;  2 usages
    private final CategoryService categoryService;  2 usages

    public WebController(ArticleService articleService, CategoryService categoryService) {  no usages  Volodymyr Vienko
        this.articleService = articleService;
        this.categoryService = categoryService;
    }

    @GetMapping("/")  Volodymyr Vienko
    public String homePage(
            Model model,
            @RequestParam(defaultValue = "0") int page,
            @RequestParam(defaultValue = "12") int size,
            @RequestParam(required = false) String search,
            @RequestParam(required = false) String category
    ) {
        var articlePage = articleService.getArticlesPaginated(page, size, search, category);

        System.out.println("СПРОБА ВІДМАЛЬОВАТИ СТОРІНКУ. ЗНАЙДЕНО НОВИН В БАЗІ: " + articlePage.getTotalElements());
        System.out.println("ПЕРЕДАЄМО НА ФРОНТЕНД ШТУК: " + articlePage.getContent().size());

        Map<String, String> categoryIcons = Map.ofEntries(
                Map.entry( k: "Спорт",  v: "bi-trophy"),
                Map.entry( k: "Ігри",  v: "bi-controller"),
```

```java
@RestController
@RequestMapping("/api/users")
public class UserController {

    private final UserService userService;  6 usages

    public UserController(UserService userService) { this.userService = userService; }

    @GetMapping("/{email}")
    public UserDto getUserProfile(@PathVariable String email) { return userService.getUserByEmail(email); }

    @PostMapping("/register")
    public void register(@RequestParam String username, @RequestParam String email, @RequestParam String password) {
        userService.registerUser(username, email, password);
    }

    @PostMapping("/verify")
    public UserDto verify(@RequestParam String email, @RequestParam String code) {
        return userService.verifyCode(email, code);
    }

    @PostMapping("/login")
    public UserDto login(@RequestParam String email, @RequestParam String password) {
        return userService.login(email, password);
    }
```

```java
@RestController
@RequestMapping("/api/categories")
public class CategoryController {

    private final CategoryService categoryService;  2 usages

    public CategoryController(CategoryService categoryService) { this.categoryService = categoryService; }

    @GetMapping
    public List<CategoryDto> getAllCategories() { return categoryService.getAllCategories(); }
}
```

```java
@RestController  Volodymyr Vienko
@RequestMapping("/api/articles")
public class ArticleController {

    private final ArticleService articleService;  2 usages

    public ArticleController(ArticleService articleService) {  no usages  Volodymyr Vienko
        this.articleService = articleService;
    }

    @GetMapping  Volodymyr Vienko
    public Page<ArticleDto> getAllArticles(
            @RequestParam(defaultValue = "0") int page,
            @RequestParam(defaultValue = "10") int size,
            @RequestParam(required = false) String search,
            @RequestParam(required = false) String category
    ) {
        return articleService.getArticlesPaginated(page, size, search, category);
    }
}
```

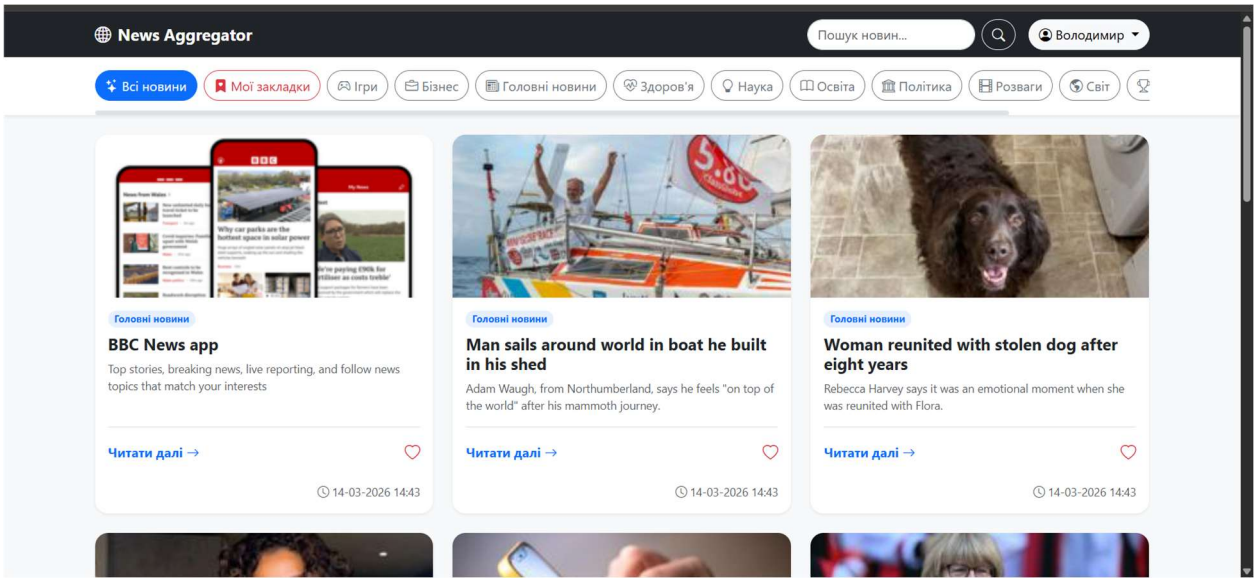
8. Результати розробки та функціональні можливості:

У результаті виконання проєкту було створено стабільний, швидкий та сучасний веб-застосунок.

Автоматизована стрічка новин та Пагінація

Застосунок автоматично підвантажує новини у фоновому режимі. Інтерфейс реалізовано за принципом "Безкінечної прокрутки" (Infinite Scroll): при досягненні низу сторінки клієнт робить асинхронний запит до ArticleController та плавно додає нові картки без перезавантаження сторінки. Зображення карток обробляються, і у разі їх відсутності підставляються естетичні заглушки (placeholder).

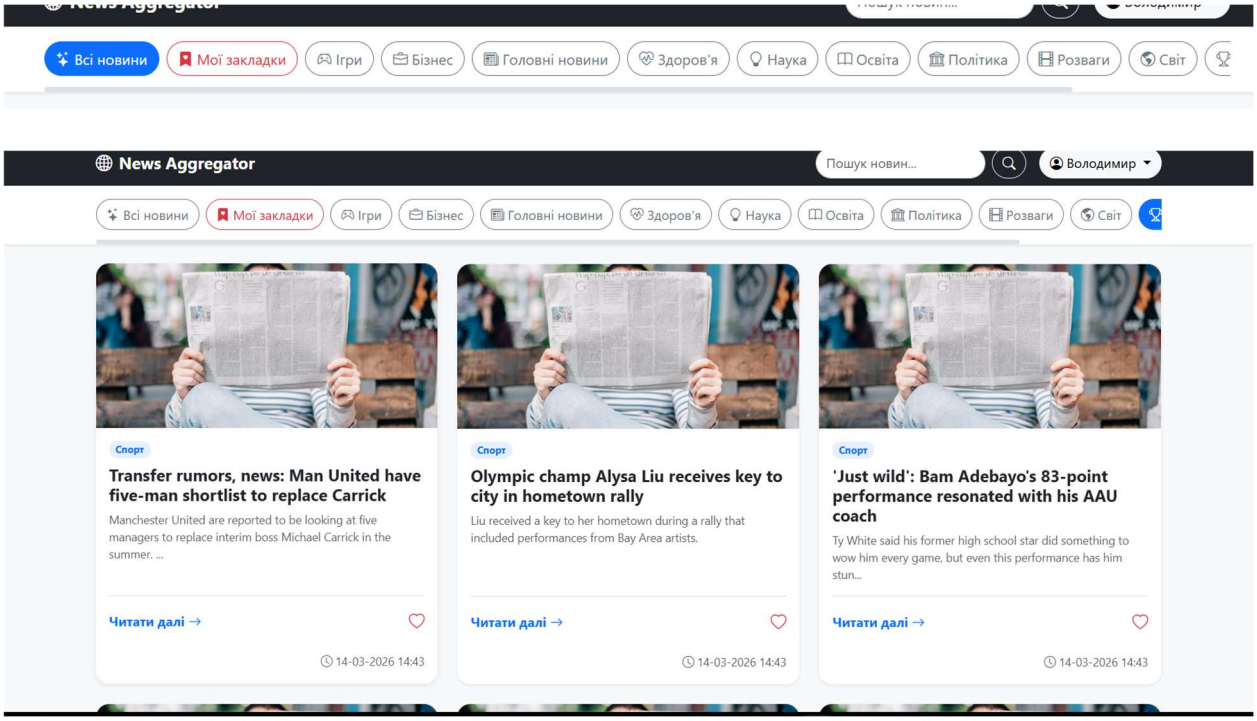
```
2026-03-14T14:43:22.135+02:00  WARN 11536 --- [news-aggregator] [           main] c
2026-03-14T14:43:22.152+02:00  INFO 11536 --- [news-aggregator] [           main] c
    Database JDBC URL [Connecting through datasource 'HikariDataSource (HikariPool-
    Database driver: undefined/unknown
    Database version: 2.3.232
    Autocommit mode: undefined/unknown
    Isolation level: undefined/unknown
    Minimum pool size: undefined/unknown
    Maximum pool size: undefined/unknown
2026-03-14T14:43:22.813+02:00  INFO 11536 --- [news-aggregator] [           main] c
2026-03-14T14:43:22.854+02:00  INFO 11536 --- [news-aggregator] [           main] j
Починаю фоновий збір новин...
Парсинг завершено! Додано нових статей: 382
2026-03-14T14:43:29.972+02:00  INFO 11536 --- [news-aggregator] [           main] c
2026-03-14T14:43:30.205+02:00  INFO 11536 --- [news-aggregator] [           main] c
2026-03-14T14:43:30.212+02:00  INFO 11536 --- [news-aggregator] [           main] c
Починаю фоновий збір новин...
```



Категоризація та Пошук

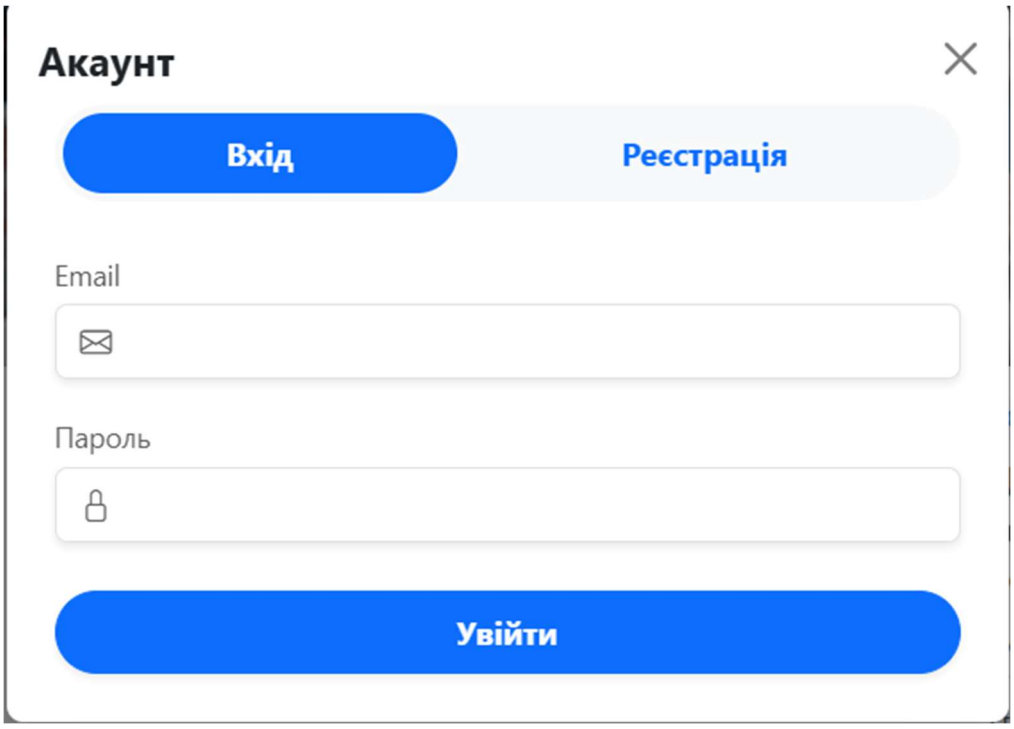
Реалізовано зручну горизонтальну панель категорій з динамічною підстановкою векторних іконок (Bootstrap Icons). Також працює "розумний" пошук, який автоматично скидає результати при очищенні поля вводу. При виборі певної категорії, наприклад "Спорт", відображаються лише відповідні

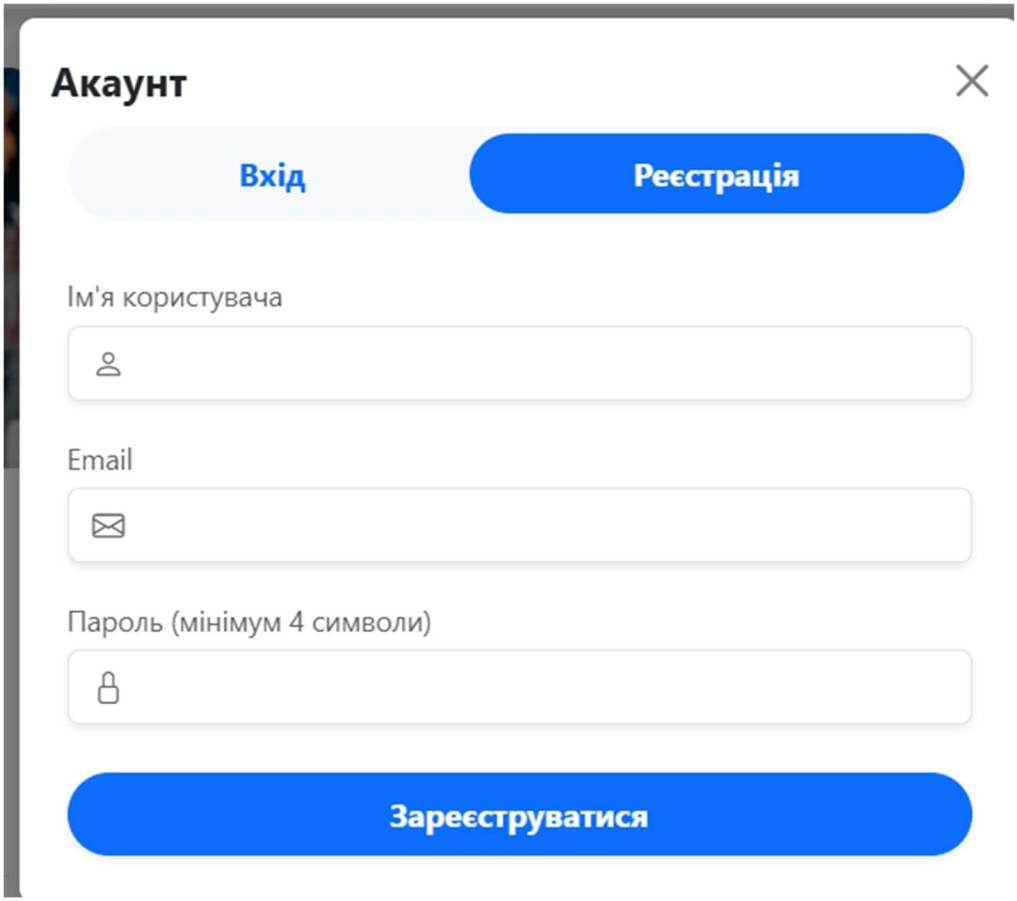
новини.



Безпечна реєстрація з підтвердженням Email

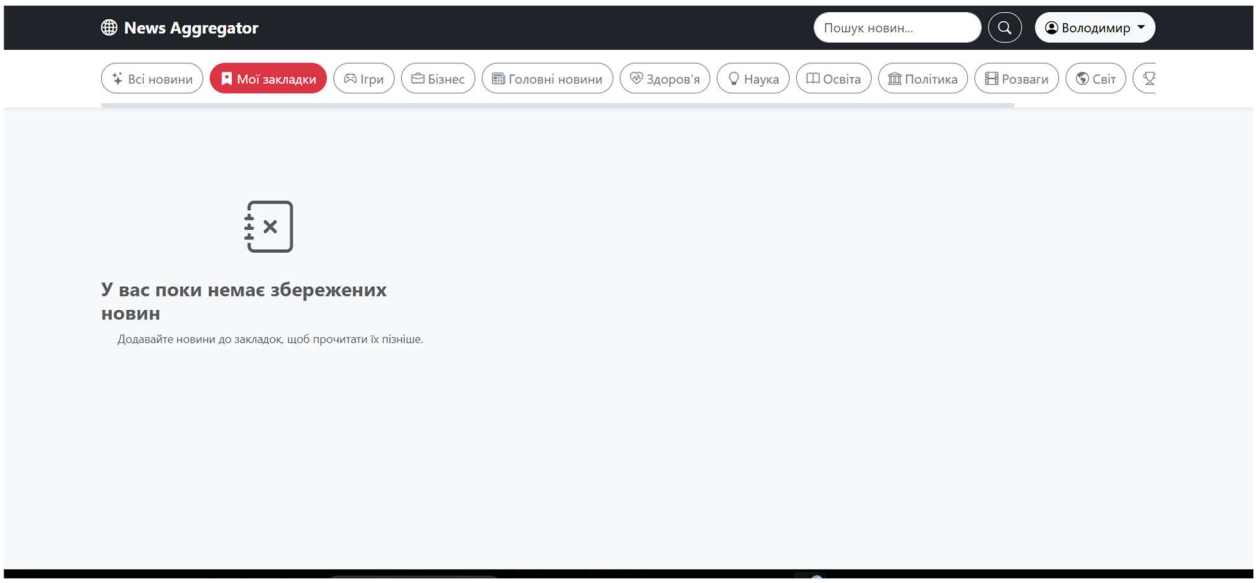
Розроблено сучасне модальне вікно з вкладками для авторизації. При реєстрації система блокує створення акаунта до моменту введення правильного 6-значного коду, який надсилається на реальну пошту користувача через EmailService.

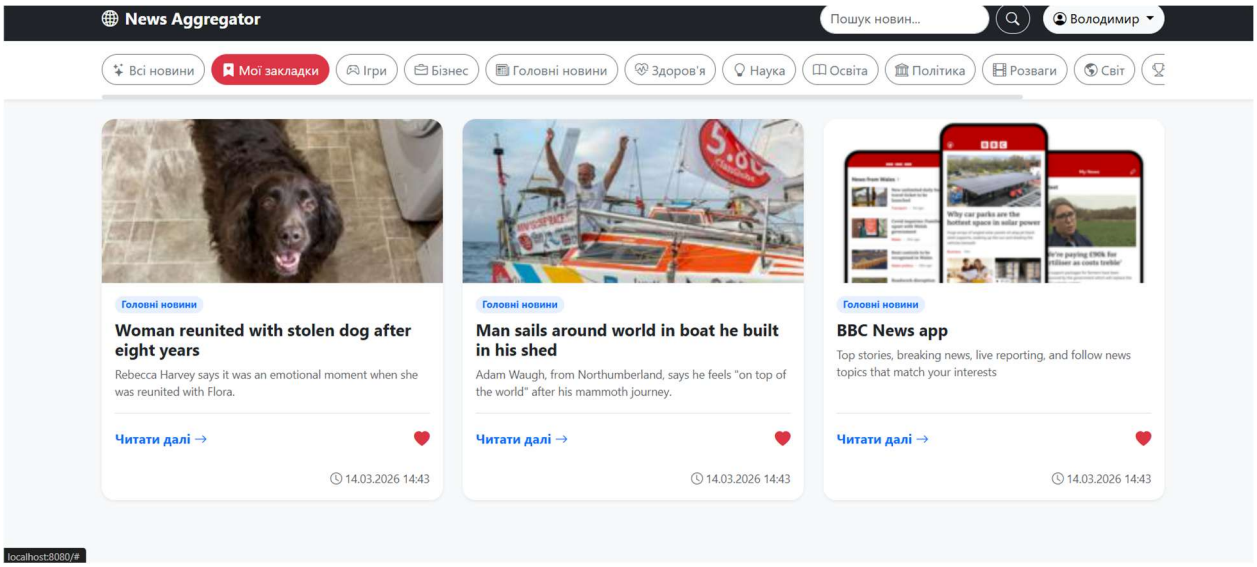




Система закладок (Bookmarks) та UI порожнього стану

Після успішної авторизації користувач отримує можливість зберігати улюблені новини, натискаючи на іконку серця. Реалізовано миттєве асинхронне оновлення стану закладок. Створено окрему вкладку "Мої закладки", де відображаються лише збережені статті. За відсутності збережених новин користувач бачить спеціально розроблений UI "порожнього стану" (Empty State).





Висновок: У результаті виконання лабораторної роботи було створено повноцінний програмний продукт «News Aggregator». Проєкт має професійну модульну структуру, розділену на шари Controllers, Services, Repositories, DTO та Exceptions.

Використання фреймворку Spring Boot дозволило швидко налаштувати REST API та взаємодію з базою даних H2 за допомогою Spring Data JPA. Впровадження бібліотеки Jsoup забезпечило автономність застосунку завдяки безперервному парсингу зовнішніх джерел. Глобальна обробка винятків (@ControllerAdvice) зробила API стабільним. Налаштування Spring Mail гарантувало високий рівень безпеки за рахунок верифікації користувачів через електронну пошту. Використання рушія шаблонів Thymeleaf у зв'язці з асинхронними запитами JavaScript (Fetch API) дозволило створити плавний та швидкий користувацький інтерфейс.